

提高作业

2.1 调研：相机内参和外参标定方法

本讲课堂中直接使用了给定的相机内参和外参。真实机器人系统中，这些参数通常需要通过标定获得。

请自行查找资料，调研相机内参标定和外参标定方法。

要求：

- 至少介绍 一种相机内参标定方法；
- 至少介绍 一种相机与机器人之间的外参标定方法；
- 说明每种方法要解决什么问题；
- 说明每种方法需要哪些输入数据或实验条件；
- 说明每种方法会输出哪些参数，例如 K、畸变参数、旋转、平移或坐标变换矩阵；
- 说明每种方法的基本流程；
- 分析每种方法的适用场景和局限；
- 绘制一张标定流程图，展示从采集数据到得到标定参数的过程。

要求：不要只复制工具说明，需要用自己的话解释“为什么这样做能够得到内参或外参”。

2.2 Aelos 实机视觉坐标转换实践

尝试在 Aelos 机器人上实现一个简化版视觉坐标转换流程。

最低要求：

- 获取一张 Aelos 相机图像；
- 使用经典视觉方法或深度学习方法得到目标 `bbox_xyxy`；
- 根据 `bbox` 计算目标像素点 (u,v) ；
- 根据相机内参或合理假设计算 `camera_ray`；
- 选择一种深度估计方法，得到目标在相机坐标系下的位置 `P_camera`；
- 根据外参或合理假设，计算目标在机器人坐标系下的位置 `P_robot`；
- 说明哪些参数是真实获取的，哪些参数是自行假设的；
- 分析这些假设会给结果带来什么误差。

如果部分参数未提供，可以自行确定一个合理值，但必须说明依据。例如：

- 相机内参来自相机参数、标定结果或合理假设；
- 目标真实尺寸来自测量或估计；
- 相机安装高度和方向来自测量或近似假设；
- 外参矩阵来自简化建模。

提交内容：

- Aelos 相机图像或截图；
- 目标检测结果图或 bbox 记录；
- 关键代码或计算过程截图；
- P_{camera} 和 P_{robot} 计算结果；
- 参数来源说明；
- 误差和局限分析。

提高作业提交要求

请将提高作业整理为 一份 PDF，包含：

- 至少一种内参标定方法的调研说明；
- 至少一种外参标定方法的调研说明；
- 相机标定流程图；
- Aelos 实机视觉坐标转换实践过程；
- 关键代码、运行截图或计算结果；
- 参数假设和误差分析。

Answers:

2.1 调研：相机内参和外参标定方法

内参决定“像素点如何对应到相机射线”，外参决定“相机坐标如何转换到机器人坐标”。两者标定准确，后续 bbox 到空间位置的计算才有意义。

(1) 相机内参标定方法：张正友棋盘格标定法

解决问题：由于相机成像过程中存在镜头的径向畸变和切向畸变，使得图像发生了“变形”。张正友标定法目的是要找出这个“变形规律”，校准后还原出真实世界的准确信息，使像素点能够被反投影为正确的相机射线。

输入数据与实验条件：一块方格尺寸已知的棋盘格标定板，多张不同姿态、不同位置的棋盘格图像；图像中角点要清晰覆盖画面中心和边缘。

输出参数：相机内参矩阵 $K=[f_x, f_y, c_x, c_y]$ 、径向畸变、切向畸变以及每张图像对应的棋盘格位姿和重投影误差。

基本流程：使用待标定相机从不同角度拍摄多张棋盘格图像，检测角点。利用检测到的角点坐标建立棋盘格平面坐标到图像像素的投影关系，之后先线性估计初值，再用非线性优化最小化重投影误差，最终得到相机内参矩阵、畸变系数等。

为什么有效：棋盘格角点的三维几何关系已知，同一相机内参在所有图片中保持不变；多姿态图像提供了足够多约束，可以把焦距、主点和畸变从观测误差中解出来。

适用场景和局限：适合普通单目/RGB 相机离线标定；但要求角点检测稳定、标定板尺寸准确，若姿态太少或只在画面中心采样，边缘畸变会估计不准。

(2) 相机与机器人外参标定方法：手眼标定

解决问题：求解相机坐标系与机器人基坐标系或末端坐标系之间的刚体变换 T (位姿)，包括旋转矩阵 R 和平移向量 t 。

输入数据与实验条件：机器人在多个已知位姿下观察同一个标定板，每次记录机器人末端位姿，同时通过相机图像估计标定板在相机坐标系下的位姿。

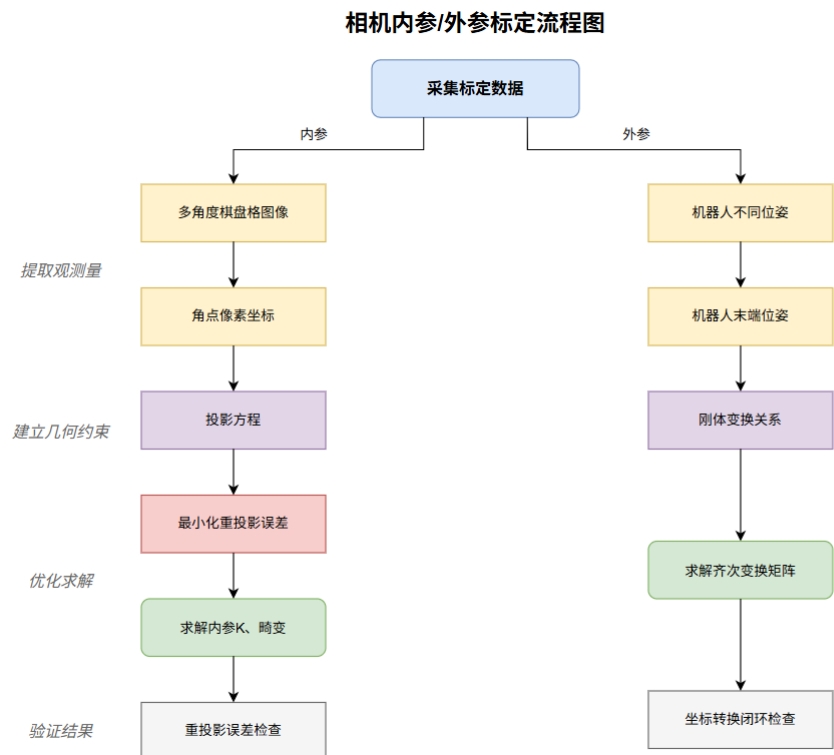
输出参数：输出齐次变换矩阵，包括旋转矩阵和平移向量。

基本流程：先完成相机的内参标定，再固定标定板或相机，采集多组机器人位姿和图像。先由内参解出标定板相对相机位姿，再构造 $AX=XB$ 或等价约束求解固定外参，最后用新姿态验证误差。

为什么有效：相机和机器人之间的安装关系是刚体常量。机器人每移动一次，相机观测到的标定板位姿会随之改变，可以通过多组运动方程解出外参。

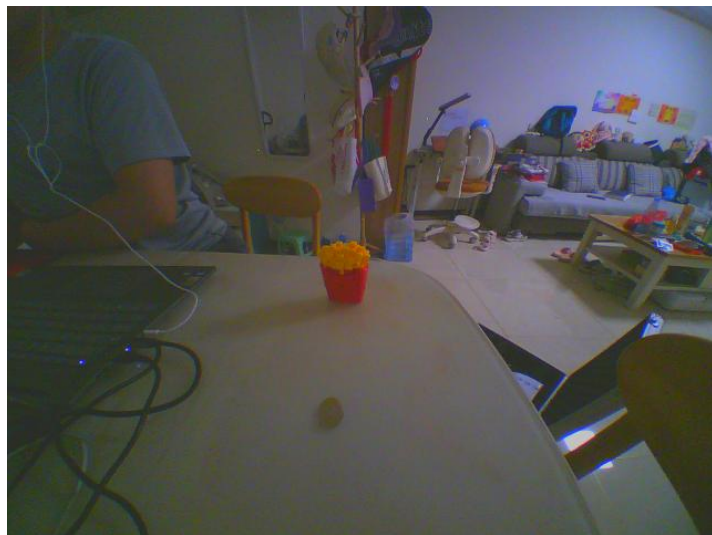
适用场景和局限：适合机械臂、移动机器人头部相机、机器人胸部相机等固定安装场景；局限是依赖于精确的相机内参和运动学模型，运动学误差会积累至标定结果。

(3) 标定流程图

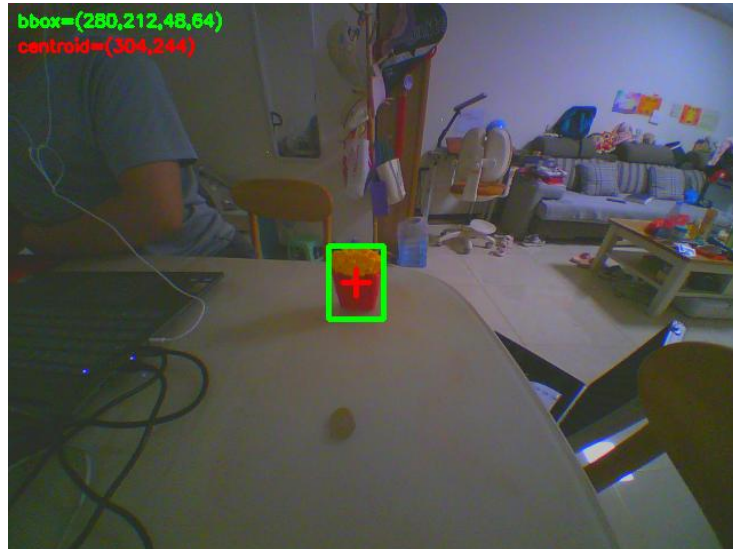


2.2 Aelos 实机视觉坐标转换实践

(1) Aelos 相机图像



(2) HSV 颜色分割后的目标框与中心点



图中的 bbox 是 OpenCV 格式(x,y,w,h)，即左上角坐标加宽高。bbox=(280,212,48,64)，centroid=(304,244)，进一步转换为 bbox_xyxy=[x1,y1,x2,y2]=[280,212,328,276]，其中右下角由 x+w、y+h 得到。

(3) 目标像素点 (u,v)

(u,v)即为 centroid=(304,244)

(4) 计算 camera_ray

在 Aelos 机器人内部获取到相机内参和畸变系数。K 矩阵为 fx=228.482, fy=229.544, cx=318.591, cy=236.217，畸变模型为 plumb_bob，畸变系数如截图。

```

lemon@raspberrypi:/mnt/leju_data/camera_info $ ls -l
total 8
-rw-r--r-- 1 lemon lemon 610 Jan 1 1970 chest_camera.yaml
-rw-r--r-- 1 lemon lemon 595 Jan 1 1970 head_camera.yaml
lemon@raspberrypi:/mnt/leju_data/camera_info $ cat chest_camera.yaml
image_width: 640
image_height: 480
camera_name: head camera
camera_matrix:
  rows: 3
  cols: 3
  data: [228.4822441704471, 0.0, 318.5911243382726, 0.0, 229.54386183804493, 236.21719592514174, 0, 0, 1]
distortion_model: plumb_bob
distortion_coefficients:
  rows: 1
  cols: 5
  data: [0.10014561706757194, -0.059807299546012965, -0.0032574413952828236, 0.00012564072537508912, 0]
rectification_matrix:
  rows: 3
  cols: 3
  data: [1, 0, 0, 0, 1, 0, 0, 0, 1]
projection_matrix:
  rows: 3
  cols: 4
  data: [228.125244140625, 0.0, 318.0933262486069, 0.0, 0.0, 238.08352661132812, 233.20433171864352, 0, 0, 0, 1, 0]

```

camera_ray=[(u-cx)/fx,(v-cy)/fy,1]=[-0.0639,0.0339,1.0000]，目标接近图像中心。

(5) 位置 P_camera、位置 P_robot

深度估计采用已知目标尺寸法 (known_size)。实际目标高度 H_real=0.12m、检测框高度 h=64px，因此 $Z = fy \cdot H_{\text{real}} / h = 229.544 \cdot 0.12 / 64 = 0.430\text{m}$ 。

目标在相机坐标系下的位置 P_camera=[Xc,Yc,Zc]=[-0.027,0.015,0.430]m (X 向右、Y 向

下、Z 向前)。

机器人坐标转换采用简化外参: $R_{robot_camera} = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$, $t = [0.03, 0, 0.32]$ m。求得 $P_{robot} = R \cdot P_{camera} + t = [0.460, 0.027, 0.305]$ m。物理含义是目标约在机器人前方 0.46m, 略偏机器人左侧 0.03m, 中心高度约 0.31 m。

项	计算结果
camera_ray	$[-0.0639, 0.0339, 1.0000]$
P_camera	$[-0.027, 0.015, 0.430]$ m
P_robot	$[0.460, 0.027, 0.305]$ m

(6) 分析

目标实际高度通过测量获取, 相机内参由 Aelos 机器人内部获取, 外参是自行假设的。外参中平移向量 t 是在 R_{robot_camera} 被简化条件下, 人为调整的, 向实际值靠拢。

目标尺寸误差: 已知尺寸法中 Z 与 H_{real} 成正比, 目标高度测量值偏差会直接变成深度偏差。

畸变误差: 本次计算 camera_ray 时, 未先去畸变。当目标接近中心时影响较小, 边缘目标则需要先矫正。

外参误差: 胸部相机相对机器人基坐标的位置和俯仰角未实测, P_{robot} 只是简化坐标转换结果。

(7) 代码参见 [main.py](#), 关键代码和运行截图如下。

```
cv2_img_cam = get_img(camera)
cv2_img = cv2.cvtColor(cv2_img_cam, cv2.COLOR_BGR2HSV)
cv2_img = cv2.GaussianBlur(cv2_img, (3, 3), 0)
contours = colour_Silhouettes(cv2_img, 1, 87, 113, 178, 255, 183)
if len(contours) > 0:
    x, y, w, h = cv2.boundingRect(contours[0])
    # 计算目标像素点(u,v)
    u = x + w / 2.0
    v = y + h / 2.0
    print("目标像素点(u,v):", (u, v))
    # 获取检测图像
    vis = draw_result(cv2_img_cam, (x, y, w, h), (int(u), int(v)))
    save(OUT_ROOT, IMG_DECT_CHEST if camera == "chest" else IMG_DECT_HEAD, vis)
    # 计算目标 bbox_xyxy
    bbox_xyxy = (x, y, x + w, y + h)
    print("目标 bbox_xyxy:", bbox_xyxy)
    # 计算 camera_ray
    camera_params = load_yaml(HERE / "camera_info" / "chest_camera.yaml")
    fx = camera_params["camera_matrix"]["data"][0]
    fy = camera_params["camera_matrix"]["data"][4]
    cx = camera_params["camera_matrix"]["data"][2]
    cy = camera_params["camera_matrix"]["data"][5]
    camera_ray = np.array([(u - cx) / fx, (v - cy) / fy, 1.0])
    print("相机射线(camera_ray):", camera_ray)
    # 深度估计
    depth_z = fy * H_real / h
    # 计算目标在相机坐标系下的位置
    p_camera = camera_ray * depth_z
    print("目标在相机坐标系下的位置p_camera:", p_camera)
    # 计算目标在机器人坐标系下的位置
    p_robot = R_ROBOT_CAMERA @ p_camera + T_ROBOT_CAMERA
    print("目标在机器人坐标系下的位置p_robot:", p_robot)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● lemon@raspberrypi:/mnt/leju_data $ python3 custom_scripts/12.2-视觉坐标转换/main.py
Run custom project
目标像素点(u,v): (384.0, 244.0)
保存图像: /mnt/leju_data/custom_scripts/12.2-视觉坐标转换/outputs/chest_camera_capture_dect.png
目标 bbox_xyxy: (280, 212, 328, 276)
相机射线(camera_ray): [-0.06386109 0.03390552 1. ]
目标在相机坐标系下的位置p_camera: [-0.02748548 0.01459276 0.43039474]
目标在机器人坐标系下的位置p_robot: [0.46039474 0.02748548 0.30540724]
● lemon@raspberrypi:/mnt/leju_data $
```